Singly Linked List – Reverse List – Space Complexity and Algorithm

Program: –

```
void reverseList(Node **head)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    Node *previous = nullptr;
    Node *current = *head;
    Node *next = nullptr;

    while (current != nullptr)
    {
        next = current->next;
        current->next = previous;
        previous = current;
        current = next;
    }

    *head = previous;
}

…….

case 14:
        {
            reverseList(&head);
            cout << "Reverse List:" << endl;
            break;
        }
```

Space Complexity Analysis

1. *Input Space*:

A. Parameter – Only Definition:

- ***The input to the function is a pointer to the head of the linked list*** ($Node\ **\ head$). ***This pointer requires constant space, i.e.,*** $O(1)$.

  This view only considers the memory for the parameters passed to the function. The single parameter `Node **head` is a pointer-to-a-pointer, which has a **fixed, constant size**. Therefore, the input space complexity is **O(1)**.

  • One parameter: head (type Node **) → a single machine word.

  ⇒ Θ(1) = O(1)

B. Full – Data (Inclusive) Definition:

This more common definition includes the entire data structure. If the linked list has **n nodes**, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

• Same single word as above (Θ(1)), plus

• All nodes that are reachable from *head when the routine is called.

If the list contains n nodes, they occupy Θ(n) bytes.

⇒ Θ(n) = O(n)

2. *Auxiliary Space*:

- ***The function uses three temporary pointers: previous, current, and next. Each of these pointers requires constant space, $O(1)$, as they do not depend on the size of the input linked list.***

  The function does not use any additional data structures that grow with the input size. It operates in place, modifying the pointers within the existing nodes of the linked list. Therefore, the auxiliary space complexity is $O(1)$.

Auxiliary space is the extra memory an algorithm uses, not counting the input. For this function:

1. **`Node *previous`**, **`Node *current`**, and **`Node *next`**: The function declares three local pointers to keep track of nodes during the reversal process. Each of these pointers occupies a constant amount of memory.
2. **Iteration**: The `while` loop reuses these three pointers to traverse the list and reverse the connections. No new memory is allocated during the process.

Since the function only needs a fixed number of extra variables (three pointers) to do its job, regardless of the list's length, the amount of extra memory used is constant.

Therefore, the auxiliary space complexity is **Θ(1)**.

• Local variables on the stack:

– Node *previous

– Node *current

– Node *next

– a few compiler-generated temporaries

Together these consume constant memory.

• No dynamic heap allocations.

• No recursion, so call-stack depth remains constant.

⇒ Θ(1) = O(1)

Algorithm

$\boldsymbol{ReverseList(START)}$

1. $[\boldsymbol{START\ is\ NULLPTR?}]\boldsymbol{if\ START = NULL, then}$:

$\boldsymbol{Write}$: `` $\boldsymbol{Error}$: $\boldsymbol{List\ is\ Empty}$. ``

$\boldsymbol{Return\ and\ exit}$.

$[\boldsymbol{END\ OF\ IF\ STRUCTURE}]$

2. $\boldsymbol{Set\ PREVPTR := NULL}$

3. $\boldsymbol{Set\ CURRPTR := START}$

4. $\boldsymbol{Set\ NEXTPTR := NULL}$

5. $\boldsymbol{Repeat\ while\ CURRPTR \neq NULL}$:

$\boldsymbol{Set\ NEXTPTR := LINK[CURRPTR]}$

$\boldsymbol{Set\ LINK[CURRPTR] := PREVPTR}$

$\boldsymbol{Set\ PREVPTR := CURRPTR}$

$\boldsymbol{Set\ CURRPTR := NEXTPTR}$

$[\boldsymbol{END\ OF\ Loop}]$

6. $\boldsymbol{Set\ START := PREVPTR}$

$[\boldsymbol{END\ of\ Algorithm}]$